

# Devpost submission — Raksha

Copy each section into the corresponding Devpost field. Replace the deployment URL placeholder once the Space is live.

**Project name:** Raksha — the real-time scam-call shield

**Tagline (≤ 200 chars):** Every scam follows a script. Raksha listens with you during a call, recognises the script in about 10 ms with Moss, and steps in before you read out the OTP.

## Links

- Deployed agent: (*Cloud Run URL, updated at submission*)
- GitHub: <https://github.com/shi1720/YC-Fall-2026-x-Moss>
- Architecture diagram: `docs/diagrams/architecture.png` (also in the README)
- PRD: `docs/PRD.md` (PDF in `docs/pdf/`)
- Video: (*YouTube link*)

**Theme:** Real-Time Voice & Conversational AI · inspired by YC RFS *Proving You're Human* and *AI for the Aging Population*

---

## Inspiration

In 2024 Indians reported ₹22,845 crore lost to cyber fraud — ten times the figure two years earlier. The "digital arrest" alone took ₹1,900 crore from 1.23 lakh people: fake CBI officers on a video call, a retired teacher told not to tell her son, her fixed deposits moved to an "RBI verification account". The Supreme Court has taken suo motu cognisance. The Prime Minister devoted a Mann Ki Baat to it.

What struck us reading the case reconstructions was how *identical* they are. Authority, fear, secrecy, then the ask. The numbers change; the script never does. And every protection on the market — Truecaller, telco spam labels, iOS call screening — is caller-ID. It tells you who is calling. Nobody tells you what they are doing to you, *while they are doing it*.

We wanted to build the thing that speaks up inside the sentence.

---

## What it does

Raksha is a shield that runs during a phone call. Every fragment of speech (even mid-sentence) is checked against a playbook of 409 real scam-script lines across 31 families, tagged with the persuasion tactic each one carries. A small risk engine credits tactics and decides *safe* / *caution* / *danger*. When pressure meets an ask, Raksha interrupts: it names the script, gives the person one sentence to say, tells them what to do, and says it out loud.

A **family code** links the phone to a guardian's dashboard: a daughter sees her mother's call risk live, can speak through the shield (her words are read aloud over the scammer's), and can ask the call's memory a

question. After the call, one tap reports the caller's lines to a community index that every running shield picks up within minutes.

Anyone can try it in 60 seconds: eighteen scripted calls with two synthetic voices (ten scams, eight genuine calls), a live-microphone mode, and a recording-upload mode.

## How we built it

---

- **Fast path — Moss, in-process.** The playbook lives in a Moss cloud index, loaded into the Node process at boot and queried for every fragment with raw cosine scores ( `alpha: 1.0` ), in roughly 10–18 ms on a shared container including embedding (the search itself is sub-millisecond). No vector database, no network on the hot path, and local queries are unmetered, so we can afford to check *everything*.
- **Risk engine.** Pure, unit-tested TypeScript: cosine → calibrated confidence; benign look-alike suppression (75 legitimate lines live in the same index so "we will never ask for your OTP" stays quiet); speaker-aware crediting; a noisy-OR over 21 tactics with persistent evidence; two hard rules — *pressure + ask* ⇒ danger, and *victim about to comply* ⇒ intervene now.
- **Slow path — LLM coach.** Only on risk transitions, a 20B open-weight model on Groq returns a JSON {`verdict`, `explanation`, `say_this`, `action`}. It never blocks the fast path and can veto a false alarm (but never override danger).
- **Moss sessions.** The process opens one `SessionIndex`; every turn of every live call is added locally, tagged with its call id; the guardian's "what did they ask for?" is a metadata-filtered semantic query over it. Turns are deleted at call end and the session is never pushed — call memory dies with the call. (We started with a session per call; opening one costs ~2 s of CPU, so 100 simultaneous calls fell over. The shared, filtered session was the fix, found by load-testing.)
- **Moss auto-refresh + multi-index.** Reported lines are upserted into `raksha-intel`; both indexes are loaded with `autoRefresh` and searched with `queryMultiIndex`, so new variants hot-swap into every instance with zero downtime.
- **Stack.** Next.js 16 with a custom server and WebSockets (one process, one URL), Tailwind, Web Speech API + Groq Whisper, Docker on Google Cloud Run, Playwright + vitest, GitHub Actions.
- **Measured, reproducibly.** `npm run eval` replays 18 scripted calls (10 scams, 8 genuine) plus 80 everyday sentences through the real engine and writes the report; `npm run eval:bench` records embed + search latency with the hardware it ran on; `npm run eval:load` drives 100 concurrent conversational calls over WebSocket (zero errors, server-side analysis p50 12.8 ms).

## Challenges we ran into

---

- **False alarms are the real enemy.** A genuine bank fraud call says "we noticed a suspicious transaction" — semantically very close to the scam. Putting benign look-alikes *in the index* and letting them out-vote tactic lines fixed most of it; making the engine speaker-aware (the victim's own words are only ever evidence of compliance) fixed the rest.
- **Reacting inside the sentence.** Final speech segments arrive too late. We analyse interim fragments every ~400 ms and let a fragment alone raise the alarm.

- **Naming the script without flip-flopping.** Courier-parcel calls *become* digital arrests. Hysteresis on the dominant family keeps the label steady.
- **Running everything in one process** so the Moss runtime, the sessions and the WebSocket handlers share memory, while Next.js bundles route handlers separately — solved with process-wide singletons.

## Accomplishments that we're proud of

---

- Retrieval + scoring in about 10 ms including on-device embedding, measured live in the app and in a committed evaluation, not claimed.
- On the committed evaluation (18 scripted calls on the real Moss runtime): all ten scam scenarios caught, none of the eight genuine calls escalated (see `docs/eval/REPORT.md`).
- An intervention a frightened 70-year-old can act on: one headline, one sentence, one action, spoken.
- Multiplayer by design — human and agent in the same session.
- A product with a buyer: from 2027 RBI makes banks compensate scam losses. Unit cost is about ₹0.02 per protected call: local Moss queries are unmetered, speech-to-text runs in the browser, and the coach makes at most ~6 small LLM calls per call (~6k tokens on gpt-oss-20b).

## What we learned

---

That latency changes *what* you can build, not just how it feels. When retrieval is free and instant you stop asking "which sentences should we check?" and check every fragment. And that the hardest part of a safety product is not catching the bad thing — it's staying quiet on the good things that look like it.

## What's next for Raksha

---

An Android app with in-call audio capture; Hindi, Tamil and Telugu playbooks; a bank-side "transfer hold" webhook; the hot path fully on-device with Moss's mobile/browser SDKs so no text leaves the phone; a deepfake-voice liveness signal fused into the risk model; and a pilot with one bank and one senior-living network.

## Built with

---

TypeScript · Next.js 16 · React 19 · Tailwind CSS 4 · `@moss-js/moss` (Moss runtime: loaded indexes, sessions, multi-index, auto-refresh) · WebSockets (ws) · Web Speech API · Groq (gpt-oss-20b coach, Whisper STT) · Zod · Motion · Vitest · Playwright · Docker · Google Cloud Run · GitHub Actions

## Team

---

Shivam Gupta (product, engineering, data) — with Claude as a build partner.